9. プロジェクトルール

> プロジェクト計画書（ミヌケル）の 9章。版 0.1（ドラフト）

9.1 命名規則

対象	規則	例
Python の変数・関数	英小文字とアンダースコア（snake_case）。関数は動詞から	`scenario_code`、`calc_rating()`
Python のクラス	単語の頭を大文字（PascalCase）。Django のモデルは単数形	`Scenario`、`RunStep`
定数	英大文字とアンダースコア	`K_USER = 32`、`MIN_GROUP_SIZE = 5`
Django のアプリ	英小文字の複数形か機能名	`accounts`、`scenarios`、`plays`、`stats`、`exhibition`
DBのテーブル	要件定義書 9.2節の名前（英小文字・複数形・snake_case）を `db_table` で指定する	`run_steps`、`victim_stats`
URL	英小文字とハイフン。URLの名前は `アプリ名:動詞`	`/scenarios/<code>/play/`、`plays:play`
テンプレート	`アプリ名/画面名.html`。部品は `_` から始める	`plays/ending.html`、`plays/_chat_bubble.html`
CSS のクラス	英小文字とハイフン。自作のものは `mk-` を付ける（Bootstrap と区別する）	`mk-phone-frame`、`mk-ending-badge`
JavaScript	変数・関数は camelCase、ファイルは英小文字とハイフン	`showClue()`、`long-press.js`
シナリオのコード	`SC-形式-番号`（形式は MAIL・SMS・SNS・WEB・TALK）	`SC-SMS-001`
シナリオの場面・結末のキー	英小文字の snake_case で、内容が分かる名前	`sms_arrived`、`end_card_stolen`
ブランチ	`種類/Issue番号-短い英語`（種類は feature・fix・docs・content・test）	`feature/12-flowchart`、`content/40-sms-scenarios`
テスト	`test_` から始め、何を確かめるかが分かる名前	`test_hide_ratio_when_less_than_5_players`
文書のファイル	`docs/` の下に英小文字とハイフン。日付は `YYYY-MM-DD`	`docs/project-plan/02-goal.md`、`docs/minutes/2026-11-06.md`

9.2 コメントと文書

- コメントは**日本語**で、「何をしているか」より「**なぜそうするか**」を書く（要件定義書 N-22）。
- 関数とクラスには docstring（目的・引数・戻り値）を書く。
- 要件に対応する処理には、要件のIDを書く。例：`# F-19-3: 分母が5人未満なら割合を出さない`

- やり残しは `# TODO(名前): 内容 #Issue番号` と書き、必ず Issue にする。放置しない。
- Claude Code が書いたコメントも同じルールで見直す。意味のないコメントや、コードを言い換えただけのコメントは消す。
- 画面の文言は、中学生でも読める短い文にする。
- 文書は Markdown で `docs/` に置く。決めたことは議事録だけでなく、該当する文書（要件定義書・本書）に反映する。

9.3 ソースコードの共有

ルール	内容
置き場所	GitHub の Atelier2006/graduation-project
main ブランチ	常に動く状態にする。直接 push しない（ブランチ保護を設定する）
作業の単位	1 Issue ＝ 1 ブランチ ＝ 1 Pull Request。PRは小さく（目安は変更400行まで）
PRに書くこと	対応する Issue 番号、変更の内容、確かめ方（スクリーンショット）、Claude Code が書いた部分とその要点
取り込みの条件	CI（Linux と Windows）が通る。作った人以外の1人以上が承認する。要件の確認方法を満たす
取り込み方	Squash and merge（1つのPRを1つのコミットにまとめる）
コミット	日本語。1行目は50文字以内で「何をしたか」、本文に「なぜ」。例: `フローチャートの割合を1本目の記録で計算する（#12）`
push	その日の作業の終わりに必ず push する（途中ならドラフトのPRにする）
入れないもの	`.env`（秘密の設定）、DBのファイル、個人情報、アンケートの生の回答（.gitignore）
競合（コンフリクト）	PRを作った人が直す。分からないときは、そのファイルを最後に変えた人と一緒に直す

9.4 Claude Code の使い方

1. 依頼は Issue 単位で出す。Issue には要件のIDと「確認方法」を書く。共通のルールは `CLAUDE.md` に書く。
2. Claude Code が書いたコードは、**担当の人が読んで説明できる状態にしてから**取り込む。PRに「説明できる」のチェックを入れる。
3. 数字・出典・事実（統計や手口の説明）は、人が出典を開いて確かめる。
4. 秘密の設定値、個人情報、アンケートの生の回答は渡さない。
5. 利用制限に備えて、締め切りの直前に大きな依頼をしない（R18）。

9.5 完了の定義（Done）

作業は、次をすべて満たしたときに「完了」とする。

- [ ] 要件の「確認方法」を満たす
- [ ] 自動テストを書き、CI（Linux・Windows）が通る
- [ ] 作った人以外がレビューして承認した
- [ ] 関係する文書（要件定義書・設計書・README など）を直した
- [ ] スマホ（幅360px）と PC で画面を確かめた

9.6 タスク管理

- GitHub Issues のラベル：`feature`（機能）、`bug`（不具合）、`docs`（文書）、`content`（シナリオ・データ）、`risk`（リスク）、`critical-path`（クリティカルパスの作業）。
- GitHub Projects のボード：未着手 → 作業中 → レビュー待ち → 完了。作業中は1人2件まで。
- Issue には、WBS の番号、担当、期限（最遅終了日）を書く。

← 08-risk.pdf | 表紙・目次 | 表紙 →